R cribsheet: CKMR course, NOAA, Hawaii, 2026

Mark Bravington: updated 2026

The course doesn't require you to run any code; it's fine to just think "conceptually", if you prefer. But if you do want to understand the code and how to write your own, then you'll need this document. For this 2026 course, I've updated the materials substantially, but the structure is pretty much the same as in previous courses.

1. You probably got this document from `https://github.com/eriqande/hi2026` or `https://github.com/markbravington/hawaii-ckmr-26`. The latter is where Mark's bit of the course stuff (including most of the R code) will live, *apart* from a few R packages whose (simple) installation instructions are given here. Please try to install those packages *before* the workshop[1], since I'm not planning to run any "software setup" sessions. Also, try to have a read of the rest of this document shortly before the course— but there's no need to study it in great detail. And if you get totally confused, don't worry— it's partly there as an aide-memoire for afterwards.

2. I recommend R version 4.6.x, but 4.5.x will *probably* work fine (I haven't tested it recently).

3. If you copy any bits of R code from this document, it will probably contain extra spaces and linefeeds and weird PDF stuff, so you can't paste it directly into R— you'll have to (slightly) clean it up in a text editor first. What you get seems to depend on which PDF viewer you use. It's annoying— sorry about that, but I don't know how to fix it[2].

4. You need to install some R packages, some from CRAN and some (of mine) from R-universe[3]. Fingers crossed, all you'll have to do is this:

```
options( repos=unique( c(
    'https://markbravington.r-universe.dev', # my stuff
    getOption('repos'),                      # your normal place
```

[1] Only if you are actually planning to run R code yourself. And if you *really* get stuck with installation, you can email me :)

[2] According to StackOverflow, the Latex package "listings" has a 60 page manual— maybe there is a solution somewhere in there...

[3] I long ago gave up on CRAN for distributing my own packages; it's a huge PITA for package writers. Although there may still be versions of some of my packages on CRAN, you should *not* install them; they'll be massively out-of-date. Please use the R-universe ones instead.

```
    'https://cloud.r-project.org'            # emergency version of CRAN
  )))
# Some from CRAN (will probably auto-install a couple of others)
install.packages( c( 'doParallel', 'RTMB'))
# IFF that succeeds, then ...
install.packages( 'mvbjoblot') # should install all my needed packages
```

If there are no error messages, and if you can now successfully do `library('offartmb')` and[4] `library('despack')`, then go to step 6.

5. Paranoia time: *you probably not will reach this step, and you do not need to read it unless you are unlucky*. If you do see error messages during installation, indicating that some of those packages *weren't* installed... oh dear.

   (a) The CRAN/base-R packages `Matrix`, `Rcpp`, `TMB` do not always play nicely with each other, cozza version problems. Sigh. You might need to re-install them in that order, the latter two perhaps from source (ie not as binaries) if that's what the messages say. Package `Rcpp` and `Matrix` are essential, but if you really can't install `TMB` or `RTMB` it's not the end of the world course-wise. FWIW, I avoid `update.packages` and prefer to stick to `install.packages`. All this is standard, albeit confusing and annoying, R stuff, so I'm not going to go into it further.

      i. I have had, ignored (since I had no idea what to do...), and survived, `Matrix`-related warnings of this type:
      ```
      Warning in .recacheSubclasses(def@className, def, env):
      undefined subclass "ndiMatrix" of class "anysparse";
      definition not updated.
      ```

      ii. For The Record: the non-default CRAN packages required for the whole shebang are `doParallel, foreach, iterators, Rcpp, RcppEigen, RTMB, TMB`. Somewhere in there (and the perpetually annoying `Matrix`) is where version conflicts are most likely.

   (b) The only packages of *mine* that you absolutely *need* to have installed to run most of the course stuff are `mvbutils`, `atease`, and `offarray`. If you can `library(offarray)` without problems (it autoloads the other two), then you can get by until Design, on Thursday. For Design, you also need `microscoping` and `despack`.
   Of the rest, it's more important to understand *why* they're being used in the course, and watch their results, than to run them yourself. For The Record:

      i. `deconvodisc` is used in a couple of examples, but you may never need it yourself.

      ii. `offartmb` is only required to fit some of the examples using `RTMB` rather than "natively"; that's entirely optional.

[4] These packages have the most dependencies, so if you can load these then you can definitely load the rest.

iii. `mvbjoblot` exists only so that all my packages can be installed with a single call, as above.

iv. And `debug` (Section 6) might hugely improve your R life, but you can elect to suffer without it; millions do. But then, millions of people vote for evil morons, too...

6. **Hooray!** There used to be more instructions, but now there aren't. Progress!!!

1 Organization of R stuff for the course

Data lives in the "datasets" folder, scripts/functions live in the "r-scripts" folder. However, it's worth having a read thru *this* document, to understand a bit about the structure of the R code. Sections 4 and 5 describe some less-well-known R features that are crucial to the scripts. Don't worry if those bits don't make sense yet.

- The course code all sits in a few script-files with extension dot-r, and the data in "dot-rda" files (which can only be loaded into R— e.g. they are not spreadsheetable). The structure is described in section 3 below.
- Most of the work is done by example-specific R functions[5], not by line-by-line commands in the scripts— the scripts mostly call the functions. When it comes to it, your core job as a CKMR "modeller" is to write *one* function— a log-likelihood. You'll probably need to write some data-organization stuff too, but that's fairly standard R.
- The script "ckmr_funs.r" *creates* a bunch of functions that are referred to in the other scripts— you can just `source()` that one without trying to read it immediately. Reading can come later.
- Each other script pertains to one specific dataset/example, e.g. "fit_delfi_A.r". You're meant to cut'n'paste lines and intersperse your own, especially for diagnostics/output after the actual fitting step— *not* to just `source()` each script without reading it, because that would teach you absolutely nothing. Each script prepares & fits the data for one example; see below for more.
- This document (tries to) refer to R *scripts* and other files via normal text in quotes, e.g. "fit_notog.r". R *functions* and other R objects (and sometimes the names of R packages, but not always) get monospaced courieresque font like this: `lglk_notog()` and `env` (and `despack`). I try to always put parentheses after a function name, so you know it's a function— but I may have slipped up here and there. If you don't understand the difference between scripts and objects, and between functions and other objects, you should get your Google boots on right now!

[5] I.E.: I wrote these particular ones, but by the end of the course you're supposed to be able to write them yourself.

- "IMOAL" <=> "In My Opinion, At Least"
- Sections 4 onwards are about R itself: stuff you mightn't know but that is used in the course.

2 IdiosyncRasies

While I haven't set out to write obscure R code[6], everybody's R experience is different and there's no one best/clearest/standardest way. So I've just written stuff that *I* think is reasonably clear, and efficient enough to run in real-time— and, crucially, stuff that *I* can follow. It's commented, but succinctly; IMOAL having too many mid-code comments is counterproductive. If you are expecting to grasp the coding of CKMR— something which can get a bit complicated— then you had better be able to fight your way through (what *you* see as...) the idiosyncrasies of my R code!

The only bits you *really* need to follow, I reckon, are the log-likelihood functions (with `lglk` in the name). You should also be to follow the "fit_<blah-species>.r" scripts— they're simpler, I think— and if you are keen you *could* look at the `boring_data_prep_<blah>()` functions (Eric may provide alternative versions of some of those).

A couple of tips for deciphering function code (tips you probably won't see in Proper Books):

- Don't necessarily start at the top. If you get a bit stuck, just look vaguely for statements in the *middle* that might do something interesting and recognizable, and focus on those. Then work *backwards* to see how they were set up, and *forwards* to see what comes out at the end.
- If you don't understand a line in detail because it uses "unusual" syntax/functions, then just *ignore* it (for now, or for ever!). It's likely to be operationally crucial, but only in a nuts-and-bolts way that is not "conceptually" important. Eventually you can sort out all the grisly details if you really have to, but don't *start* by trying to understand every minor operational nuance.

That said, there are a few things about the course code that might benefit from a few words. This stuff is *R-specific detail*, rather than interesting stuff about CKMR, so— as per above— you should not start by trying to memorize it all. Whizz thru it quickly now, and return to it only during/after the course when you are flummoxed by something.

- Where do the "data" and "useful byproducts" live/get kept, inside each `...lglk...()` function? They live in its *`environment`*; more on that below.
- Notation: I am not 100% consistent here, and sometimes favour aesthetics or brevity or laziness over convention, but:

[6] Despite what some of my former colleagues think.

  - "Data" and "pop dyn quantities" often start with capital letters; my general housekeeping variables usually don't. (The initial letter "n/N" breaks this "rule".) I use underscores a lot, and I never use camelCaseBecauseIHateIt, except as noted next.
  - for (off)arrays [see below], I usually embed the "names" of the subscriptors/dimensions at the end, eg n_comp_POP_BYA will be 3D with dimensions B(irth-year), Y(ear—of sampling), A(ge at sampling).
  - Probabilities start with Pr_, and are mostly of the form Pr_<things>_<givens>, so that Pr_AtruB_CDE is the probability of "Atru" and "B" each taking a specified value, given values for "C", "D", and "E"; it will be a 5D array.

- offarray: just like a regular R array (or matrix or vector), except that dimensions can start at any number, including negative (regular R arrays must start at 1). Very handy for population dynamics starting in year 2022 at age 5! Character-indexed dimensions are also allowed, e.g. SEX=c('F','M'). I find offarray a lot more natural for "real data" and I make less mistakes. Things basically "just work", but there *are* some differences from base-R behaviour (even if your index happens to start at 1)— see ?offarray in the eponymous package for more info.

- Loops: instead of nested for-loops, you'll see heavy use of autoloop() instead, like so:

```
thing_XY <- autoloop(
    x=2020:2028, y=5:15,
   yeareff[ x] * exp( -zeddo * y)
)
# ... is like doing this in mostly base-R:
thing_XY <- offarray( 0, dimseq=list( X=2020:2028, Y=5:15)))
# ... made an empty; dimseq specifies "dimensions"
for( x in 2020:2028) {
  for( y in 5:15) {
    thing_XY[ x, y] <- yeareff[ x] * exp( -zeddo * y)
}}
```

  autoloop() is much faster than explicit loops, particularly for offarray objects. You can also do "sum-contractions" as per e.g. a matrix-multiply. See ?autoloop in package offarray. It's also fast in RTMB, whereas normal nested for-loops are slow (sometimes to the point of being unusable in complicated stock-assessment models), unless you use tricks.

- Package atease lets me write x@a instead of attr(x,"a") and x@a<-<something> works too. This works for any x, not "just" S4 objects (which is what @ is normally reserved for in R[7]). Just convenient.

[7] The advent of R package S7, as the love-child and possible successor of S3 and S4, means that I ought to Islightly change atease at some point, but it will still work.

- `cq( some, words, here)` is exactly like `c( 'some', 'words', 'here')` but looks less quoty.
- `returnList(a,b)` puts `a` & `b` into a list with names `"a"` and `"b"`. I often use it to simulate the "multi-argument returns" that R used to have.
- `extract.named( <list with names a, b, c>)` will create the objects `a`, `b`, `c` in the place it is called from.
- You don't generally need to see inside the `boring_data_prep_<blah>()` function for each example, but if you do you might see some package `mvbutils` operators such as `'%where%'` (that particular one is for subsetting dataframes by rows). They are documented, and/but kinda obvious.
- I make heavy and unapologetic use of features in base-R, some of which may be unfamiliar (but they are documented in base-R, so that's not my problem...). They include: `environments` and lexical-scoping (see section later on), `local`, `with`, `evalq`.
-
- I haven't used any tidyverse stuff, and to me it's a poor fit with CKMR; it makes it really hard to do the probability-blurring steps that are crucial in realistic CKMR projects, whereas they are succinct and obvious with `offarray`. However, tidyversistas can make the simplest and least-realistic examples cases work easily enough , and some people might prefer tidyverse for the data-preparation steps (it's really up to you how you write those, but in the end you need to produce an array or offarray, not a dataframe).

2.1 Optional: using `RTMB` or `TMBO`

The examples in the course are deliberately simple, in part so that the original R versions can be fitted to data with tolerable speed. However, real-life cases are often more complicated, e.g. with lots more parameters and with random effects, and R-only code just won't be quick enough. You have two choices: rewrite in some version of TMB/C++ (see below— but almost no-one chooses to do this now), or just use the `RTMB` package (see next). Note that there is *absolutely no need* to do so during the course, unless you Like That Kind Of Thing.

Kasper Kristensen's `RTMB` package is a truly amazing thing; I am in awe! It lets you leverage the power of `TMB` without having to write (and debug) ghastly C++ code!!! If you code in the right way[8], then I *think* it may be fast enough to cope with realistically big and complicated fisheries models— by now, others will know more than me. In 2024, I painfully rewrote my `offarray` package to make sure it works nicely with `RTMB`. So now you can fit all the course datasets either with "native R" (just calling `nlminb` on the basic function), or with `RTMB` (by calling `MakeADFun` first). All you need to do is:

[8]But not if you don't... there are tricks.

- also load `library(offartmb)`, to set up some invisible helper functions;
- make sure your function bodies are wrapped in calls to `reclasso()`— as per the examples. `reclasso` is harmless if a function isn't being called by `MakeADFun`, but essential if it is. If you see calls to `reclasso()` sprinkled around in the code, that's why.

One `RTMB` thing I haven't done yet, is use `ADREPORT` to get derivatives of various things. (I have only just been shown how it works, and there are a few details I will need to sort out.) So, for variance calculations and so, I use "the delta method" manually, using numerical differentiation.

2.1.1 TMBO

(This is *probably* obsolete in 2026, unless you really hit speed problems with `RTMB` that aren't solved by offarrays.) If you prefer to write CKMR code in TMB-style C++, then my package `TMBO` (from the same repo) extends the C++ syntax to be fully compatible with `offarray` in R; your C++ arrays can start anywhere you like (and the same place as their R aliases), not at stupid 0. Thus it's really easy to rewrite the course examples, or more complicated extensions, in C++/`TMBO`. `TMBO` is as fast as `TMB` itself, and also *much* more helpful with debugging messages. I spent several months in 2022/2023 writing `TMBO`, and then `RTMB` happened, which probably makes it obsolete— but it's a still bit too soon to make that call. I haven't covered `TMBO` in the course at all, but can say more on request.

3 Structure of code and data

You first need to `source()` the master script file "ckmr_funs.r" to create all the functions that will be used in the various examples.

The core of each example is a log-likelihood function called something like `generic_lglk_<blah>()`. That's the one piece of code you really need to understand for that example— and ultimately to be able to write yourself for your own data.

The actual fitting of CKMR model(s) to example `<specific-eg>` is done by the *script* in file "fit_<blah>.r". The steps are:

1. `load()` the data for that example into the R session ;
2. Summarize the data into a form suitable for CKMR models (eg adding up the number of kin-pairs and comparisons by "type", ie individual covariate values and nature-of-kinship)— by calling `boring_data_prep_<blah>(...)`. Basically it's calling `table()`, to tabulate the numbers of kin-pairs and of pairwise comparisons according to various covariates. It also sets the ranges of those covariates, such as years and ages. It's got that name `boring...`

for a good reason. People might choose to write these functions somewhat differently than I did. You can look at the `boring...` code later if you want, but I suggest that it's better initially to concentrate on CKMR! The summaries get stored in an environment, called e.g. `env` — so that e.g. `env$n_MOP` has the MOP-counts.

3. Make a copy of the generic lglk that will be specific to this example, called something like `lglk_<specific-eg>()`.

   (a) The reason you have to make a copy is that `generic_lglk_<blah>()` mentions the data by name (eg `n_MOP`) but can't be called as-is, because it hasn't been told what those data actually are for *this* example. And you might well be able to re-use the same generic in different examples, so I don't want to tie it to a single dataset from the get-go.

4. "Tell" your copy `lglk_<specific-eg>()` about the specific data, by resetting its environment to `env`. (This is fundamental R stuff, but lots of people don't know about it— see section 5, and/or google "R lexical scoping".)

   (a) Whenever `lglk_<specific-eg>()` gets called, it will now update various intermediate quantities such as the fitted numbers-at-age, which are also stored in `env`. They will stay put in `env` after the call has finished. That's how you get at estimated quantities-of-interest (such as the array of numbers-at-age-and-year) after actually fitting the model.

   (b) Note that `env` and `environment(lglk_<specific-eg>)` are now *identical*— they refer to exactly the same bit of memory and they're not copies of each other. Changing something in `env` will also change it in `environment(lglk_<specific-eg>)`, and conversely.

5. Pick starting values and fit the model using `nlminb()` or `optim()`. As I said, you've now got access to eg estimated numbers-at-age-and-year matrix inside `env`.

6. Tiddle around with the output in various ways: plots, diagnostics, covariances;... All based on stuff in `env`.

3.1 Data

The data for example "<blah>" lives in the file "samp_<blah>_raw.rda", which you need to `load()` (as per the script for that example) to create an object `samp_<blah>` in your R session. It's a list with components `Samps` and the various kin-pairs, e.g `POPs`, `same2KPs` (basically, maternal half-sibs), `diff2KPs`. Some non-sample-related background data for the example (e.g. catches-at-age, age-at-maturity) is stored in the attribute `public` which, if you have loaded my package `atease`, you can get at via `samp_<blah>@public`. There might also be "secret true stuff" that you're *not* allowed to look at while fitting, in

```
samp_<blah>@secret # see "Attributes" for what that "@" is doing
```

The component `$Samps` will include fields `$ID` (a name), `$SEX`, `$Y`, plus any sample-specific age, length, or other data. The `POPs` etc are all two-column matrices whose entries are IDs corresponding to `Samps$ID`. Order may matter: `MOPs` should contain Mothers in the first column and Offsprings in the second, and in principle I would organize halfsibs as O(rdered)HSPs where the firstborn is in the first column. However, when ages are uncertain you may not be able to tell which is which. If I was being really consistent throughout I would call things `OMMOPs` and so on (Offspring-Mother/Mother-Offspring Pair) but consistency can get in the way of comprehensibility.

When fitting a model to <blah> data, the first thing is to reorganize the raw data into a more suitable form. The `boring_data_prep_<blah>()` functions do that. Often there are modelling decisions built into that function, which you might want to change: for example, which samples should/should not be used as "potential offspring" in comparisons, depending on their covariates. In CKMR you don't *have* to use every type of comparison (i.e. between every possible pair of samples)— part of the art is deciding which ones aren't worth the trouble of trying to use. (The only rule is: you're not allowed to cheat by first doing a comparison and afterwards deciding whether to retain it based on its outcome!) I have sometimes used booleans called `offposs` and `parposs` for that kind of thing, which you'll see inside `boring_data_prep_<blah>()` functions. We actually use a slightly more sophisticated approach to inclusion/exclusion in real examples, e.g. for SBT; the course will explain.

3.2 How I obtained the "data"

I did an IBM simulation of each population with my R package `kinsimmer2`, using the function `simkin2()` which keeps track of ancestry. Then I sampled from its dead ones and picked the kin-pairs in the sample, using `sample_from_catch()` [or `sample_from_all()` for non-lethals], nnd saved the result into a file "samp_<blah>_raw.rda". Package `kinsimmer2` is pretty fast (considering it's all in R, anyway), reasonably flexible, and (in 2026) reasonably documented; it's ultimately based on Shane Baylis' package `fishSim` (`shane.baylis@csiro.au`) but substantially tweaked and speeded up for my own sneaky purposes. There are other general-purpose simulators, too; eg Eric's `CKMRpop`, and I think `sequoia` has one. And I have a feeling that Claude will just do it for you if you ask em. *However*— as I'll explain during the course— I generally *don't* recommend simulations for routine CKMR anyway[9]!

4 Attributes

Most R objects can have *attributes* stuck onto them. Some attributes are built-in, like `dim` and `length`; but you can (usually) add your own, of any type and with any name you like. They are

[9] And that is one reason why I haven't put `kinsimmer2` on R-universe, though I can do so on request.

handy for storing "metadata". For example, the CK data pertaining to each scenario consists of a `list` with 2+ core CKMR elements: `$Samps`, `$POPs`, and/or `$HSPs`. But there's also case-specific background data, such as age-at-maturity or catch-at-age. Plus there's "secret" parameter values for the simulated dataset, e.g. the true abundances over time. The case-specific public stuff is all stored in a list called `public`, which is attached as an attribute the CKMR data list. The secret stuff is also stored in a list called, yes, `secret`, which is attached as another attribute. So, for the *Notogorgius* (fish) case we have

```
> names( samp_notog1)
[1] "Samps" "POPs"  "HSPs"
> names( attributes( samp_notog1))
[1] "secret" "public"
> names( attr( samp_notog1, "public"))
[1] "C_sya" "Amat"  "wt_a" # catch-at-age, age-at-maturity, weight-at-age
```

As you can see, base-R syntax for accessing and showing-names-of attributes is a PITA. So my tiny package `atease` lets you do this instead:

```
> atts( samp_notog1) # instead of names( attributes( samp_notog1))
[1] "secret" "public"
> names( samp_notog1@public)) # refers to attribute by name, more directly
[1] "C_sya" "Amat"  "wt_a" # catch-at-age, age-at-maturity, weight-at-age
> # Don't do this to your data! But the @-syntax makes it easy...
samp_notog1@public$C_sya <- samp_notog1@public$C_sya * 2
```

It's borrowing the "@" symbol used normally for S4 slot access. If you don't know what the latter is, my advice is: keep it that way :)

5 Environments and "lexical scoping"

The course code uses R environment objects to stash "permanent" data and "just-calculated" intermediate results required and/or calculated by log-likelihood functions. Most people don't know about R environments— you often don't really need to— but it's a really powerful fundamental feature of R, and I take advantage of it to make my code shorter and IMOAL clearer. Of course, the main point of the course code is just to illustrate how CKMR models should be programmed, *not* to understand the R minutiae. But if you really do want to understand how the code works, or even to adapt it for your own purposes, you'll need to understand how it uses environments.

An "environment" object in R is a lot like a "list": it *contains* several things, each with its own name, and you can extract/set them via e.g. `myenv$toothpaste` or `myenv[['toothpaste']]`. Environments crop up in several places. Importantly, every R function you write (or I write), comes with

its own environment— a *permanent* thing (though you can change its contents, and even replace it with a different environment). When the function is executing, it also has a *temporary* environment, which holds all the variables created while the function runs, plus the function's arguments; that temporary environment vanishes when the function concludes. If, while executing, the function sees a variable-name that's *not* in its temporary environment, then it looks for that variable in its own permanent environment (`P`, say); if not found there, it looks in the *parent environment* of `P`; if not there, then in the parent of `P`'s parent; and so on. What are these "parents", you ask? Simply: every environment has one "parent", also known as its "enclosing" environment. Eventually, this ancestor-chain (usually) reaches `.GlobalEnv`, which is the "workspace" you see directly from the command-line; that's why functions can usually get away with referring to "global variables" (though the potential for confusion is huge). In fact, if you just define a function at the command-line without taking special measures, then its permanent environment will be `.GlobalEnv` itself— a very short ancestor-chain. And `.GlobalEnv` itself has a ancestor-chain: the "search path" that you get from `search()`, which ends (basically) in the "base environement" `baseenv()` (you have to call that *function* to refer to the "base environment", there's not a reliable built-in variable— Becos R). Functions in packages have slightly different ancestor-chains.

The practical upshot for complicated log-likelihoods and so on, like all the CKMR `lglk_<blah>()` functions in this course, is that you can use your `lglk_X`'s permanent environment to:

1. stash all the constants and data that you need to compute the lglk (not the parameters, obviously; they are set thru the function's arguments when you call it), so you don't need to explicitly pass in all those data-and-constants as arguments every time you call the function;

2. store intermediate quantities computed during the lglk calculation, e.g. the population-dynamics numbers, so they can be retrieved afterwards. (They will be created in the temporary environment, but you can copy them to the permanent one);

3. and if you have several datasets all with similar structure to which you're going to fit the exact same model each time, you can make a separate copy of the "generic" function for each one, and set the permanent environment of each copy to match one particular dataset. (Don't change the environment of the generic— too risky.)

Here's an example of stashing some data:

```
> somefun <- function( x) x+mm^2 # data is 'mm'
> suppressWarnings( rm( mm)) # make sure it's not just lying around somewhere
> somefun( 1) # won't work; can't find 'mm'
Error in somefun() : object 'mm' not found

> environment( somefun) # So, where does somefun() look for stuff, by default?
<environment: R_GlobalEnv>
```

```
> zoup <- new.env()
> zoup$mm <- 7 # defining 'mm' inside environment 'zoup'
> environment( somefun) <- zoup # make it look here
> somefun( 1) # bing bong
[1] 50
```

And here's an example of storing intermediate quantities for subsequent "posthumous" retrieval:

```
> var_recip_x <- function( xvec){
+   mean_recip <- mean( 1/xvec)
+   mean_recip2 <- mean( (1/xvec)^2)
+
+   # Now gonna put both into the permanent env, for later access
+   REPORTO( mean_recip, mean_recip2) # ?mvbutils::REPORTO for how
+ return( mean_recip2 - mean_recip^2)
+ }

> zoup <- new.env()
> environment( var_recip_x) <- zoup
> ls( zoup) # nada
character(0)

> var_recip_x( 1:5)
[1] 0.08418

> ls( zoup) # oooooh!
[1] "mean_recip"  "mean_recip2"

> zoup$mean_recip # ta-daaah
[1] 0.4567
```

5.1 More on `environments`

There are a couple of good web resources on R environments: `https://adv-r.hadley.nz/environments.h` is one. And there are a few R functions specifically for environments which you should know about: e.g. for converting to/from lists. R has a help system and manuals for all that! (Try "Scope" in "An Introduction to R". And there is stuff in "R Language Definition" which may help and/or hinder comprehension.)

Environments are powerful, but there are few things to be careful about:

1. Environment chains can also make your code *spectacularly* incomprehensible, if you try hard enough. Some of the people who wrote the base-R code seem to enjoy doing that. Try figuring out how on earth `lazyLoad()` works, or how `srcref`'s are implemented. Grrrrr!

2. Unlike almost everything else in R, an environment is a *reference object*; it just "points" to a collection of stuff in memory, rather than *being* that collection. In particular, two environments with different names can actually "point" to exactly the same collection, and changing the contents of one will change the other. R just doesn't do that with anything else! (Though C etc do.) So the following code might be a surprise:

```
A <- new.env()
A$toothpaste <- 'okapi'
B <- A
B$toothpaste <- 'lambrusco'
A$toothpaste # surprise!
```

   If you changed the first line to `A<-list()`, you would *not* get the same behaviour. But don't panic: when you set up an environment, it does make an actual separate copy of anything you put into it, so e.g. this won't cause problems:

```
A <- new.env()
earwax <- 5
A$earwax <- earwax
A$earwax <- 27
earwax # still 5
```

   That is: the "collection-of-stuff" which one (or more) environments point to, can only be accessed via those environment(s), and is completely separate from any other copies of the things inside it.

3. Be careful of the distinction between the environment that you *call* a function from (usually either `.GlobalEnv`, or the temporary environment of some other function that's executing), vs. the temporary-and-permanent environments of the function itself. They are different—except if they happen to be the same ;). Does that sound confusing? Well, it is. But it's the basis of *lexical scoping.* Google is your friend here :)

6 The `debug` package

My `debug` package is to help with debugging functions— the main objective— and nowadays also scripts. You don't absolutely *need* it, but it might help your R life out a lot. And it might save a lot of public $ that you would otherwise $pend trying to get stuff right. And even if your own code is Perfect (Well Done You!!!), using `debug` might help you understand what in hell is going on with other people's monstrosities!

Plus, if you've actually followed the instructions, you've already installed `debug`. So why not have a look? `package?debug` will get you started; then follow the links, e.g. to the main function `mtrace()`.

Here are some comments I've heard on R debugging, and my responses (cleaned up for general audience):

- "*R/Rstudio has already got built-in debugging facilities, like* `browse`": Mmmm yeah kinda sorta— but they're pretty rudimentary. I had better debugging tools available when I started coding "properly" in the late 1980s, with Turbo Pascal. It's now 2026 FFS... (And C++ programmers do not know what they are missing, BTW.)
- "*I just paste statements one-at-a-time into the console to test them*": Sure, we all do that— but you can't when you're debugging a `for`-loop or `if`-statement, for example. And, when you are hoping to imitate what goes on *inside* a function but from the command line, goodness knows what leftovers might be in your `.GlobalEnv` to confuse matters.
- "*I just use* `print` *statements*": Well, you *can* light a fire by rubbing sticks together, too— maybe[10]. Or you can use matches. People in the know seem to prefer the latter.
- "*If one approaches one's coding Properly, e.g. in accordance with ISO1778-14, one will find that one has no need for a debugger.*": Ummm, well, "that's nice, dear". Meanwhile, back on Planet Earth...

It's also worth looking at Simon Wood's web-page on such matters:

`https://www.maths.ed.ac.uk/~swood34/RCdebug/RCdebug.html`. (NB it is out-of-date on a few details of `debug`; I've improved a number of things in `debug` in recent years.)

Curiously enough, this CKMR course is not an ideal demo of `debug`, because I've used `autoloop()` so heavily, and `debug` doesn't *by default* look inside `autoloop` statements (it will just report the error and pause outside the `autoloop`, just as it would with most other functions). However, you can *make* it look inside, as explained in `?autoloop` and the section on "Debugging". That said, `autoloop()` is pretty good at telling you where errors happened, and CKMR code is so structured that you may not need a debugger (much), even if you don't follow ISO17.... But `debug` sure helped when I was trying to *write* all this code in the first place.

[10] Actually it's quite difficult.